

Universidad Nacional de Ingeniería - UNI
Ingeniería en Computación

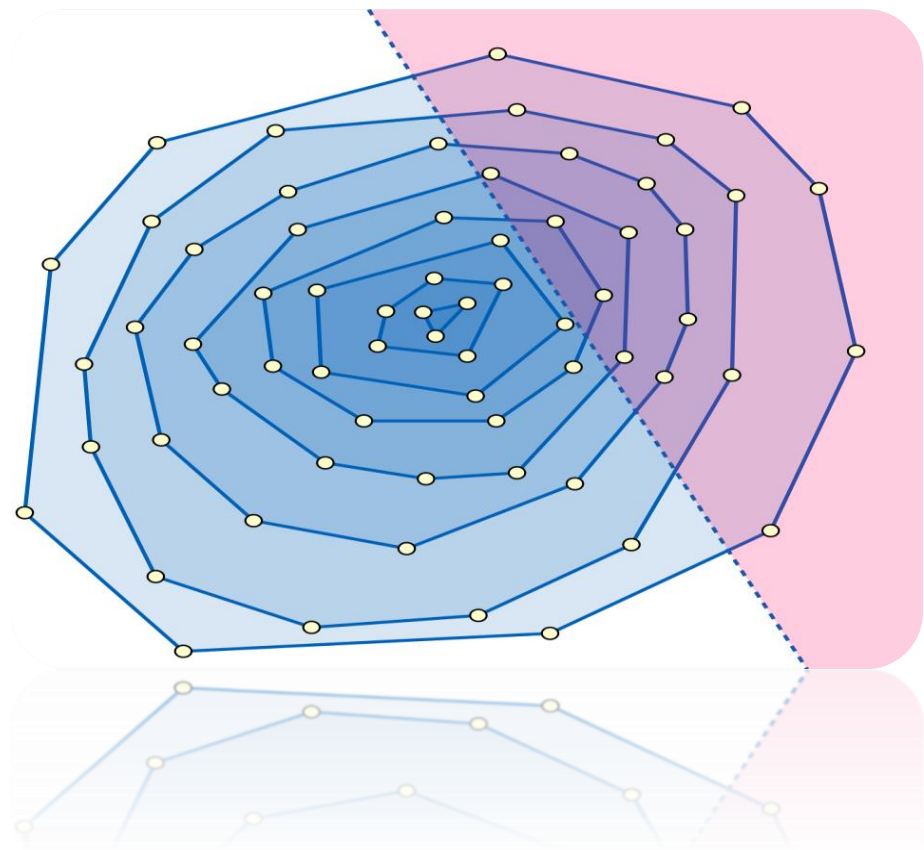
Geometría Computacional
2o año - I semestre. DACTIC/Virtual

Delimitación Perimetral usando Cierre Convexo

Autor: **Nelson Antonio Blandón**

Carné: **2017-1392U**

Docente: **Ing. Yahaira Andrea Altamirano Briceño**



1. Planteamiento del Problema

En el contexto de un proyecto de planificación territorial e infraestructura, se han registrado las coordenadas bidimensionales de diversas estaciones de monitoreo distribuidas aleatoriamente en una región de estudio. Para optimizar el despliegue de recursos y establecer un **cerco perimetral de seguridad**, es necesario determinar el polígono convexo más pequeño que encierre la totalidad de estas estaciones.

Dado un conjunto de (n) puntos en un plano cartesiano, el estudiante deberá escribir un programa en lenguaje C que procese estas coordenadas y determine exactamente qué puntos forman los vértices de la frontera (el cierre convexo).

2. Objetivo de Aprendizaje

Aplicar los teoremas de convexidad y el cálculo de orientación geométrica mediante el diseño y la implementación de un algoritmo de cierre convexo en el lenguaje de programación C, logrando la resolución algorítmica de un problema espacial.

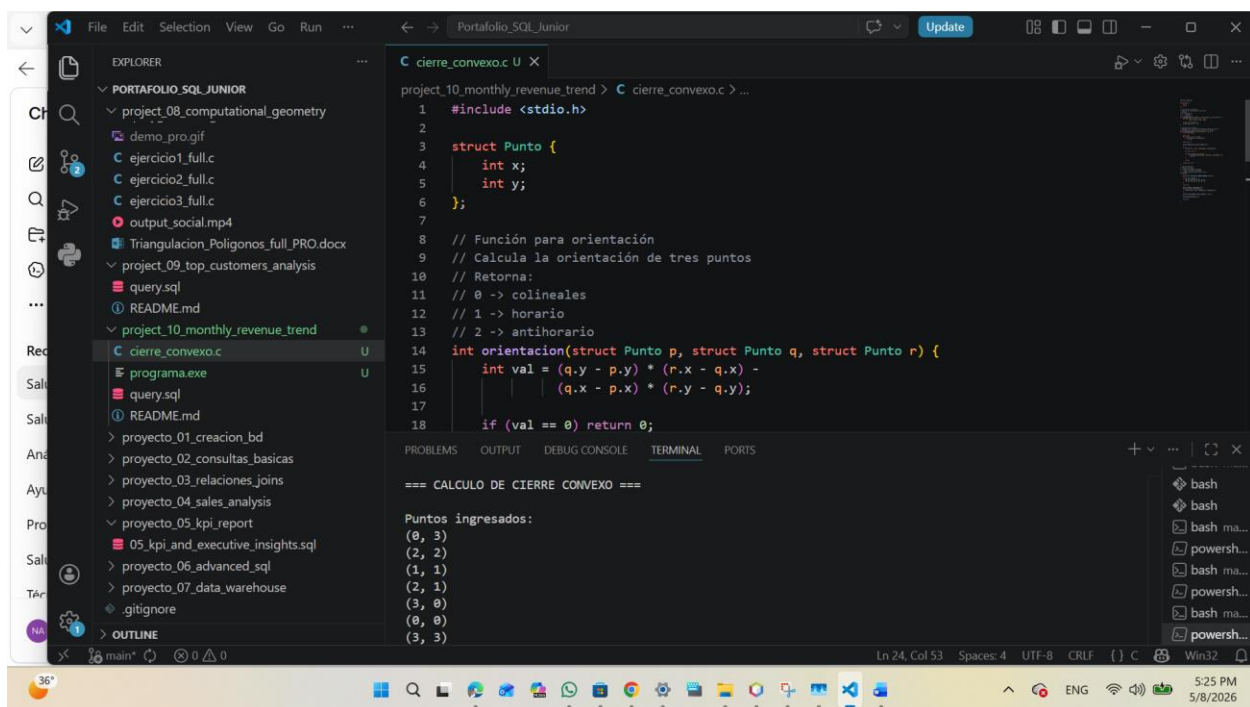
3. Pasos para Resolver la Actividad

Para completar esta asignación, los estudiantes deberán seguir los siguientes pasos:

1. **Definición de Estructuras de Datos:** Crear un struct en C para modelar los objetos geométricos, representando cada estación de monitoreo mediante sus coordenadas (x, y)
2. **Una vez obtenidos los datos de los puntos:** Dibujar polígono convexo más pequeño que encierre la totalidad de estas coordenadas de puntos, para ello se debe haber determinado con anterioridad cuales serían los vértices, según lo visto en clase, tomar en cuenta los 4 puntos importantes, (más alto, más bajo, más a la izquierda, más a la derecha)

3. **Lectura de Datos y Ejecución:** Configurar la función main para que permita el ingreso manual de al menos 8 puntos de prueba o lea las coordenadas desde un arreglo predefinido.
4. **Entrega:** Subir el archivo fuente .c al espacio de la tarea en la plataforma virtual. Recuerda también tomar captura de pantalla y subir el documento con tu nombre. El código debe estar debidamente comentado, explicando cómo la teoría matemática se traduce en la lógica de las funciones.

Evidencia de ejecución



```
File Edit Selection View Go Run ... Portafolio_SQL_Junior Update
EXPLORER
PORTAFOLIO_SQL_JUNIOR
  project_08_computational_geometry
  demo_pro.gif
  ejercicio1_full.c
  ejercicio2_full.c
  ejercicio3_full.c
  output_social.mp4
  Triangulacion_Poligonos_full_PRO.docx
  project_09_top_customers_analysis
  query.sql
  README.md
  project_10_monthly_revenue_trend
    cerrre_convexo.c
    programa.exe
    query.sql
    README.md
  project_01_creacion_bd
  project_02_consultas_basicas
  project_03_relaciones_joins
  project_04_sales_analysis
  project_05_kpi_report
  05_kpi_and_executive_insights.sql
  project_06_advanced_sql
  project_07_data_warehouse
  .gitignore
  OUTLINE
  main

cerrre_convexo.c
1 #include <stdio.h>
2
3 struct Punto {
4     int x;
5     int y;
6 };
7
8 // Función para orientación
9 // Calcula la orientación de tres puntos
10 // Retorna:
11 // 0 -> colineales
12 // 1 -> horario
13 // 2 -> antihorario
14 int orientacion(struct Punto p, struct Punto q, struct Punto r) {
15     int val = (q.y - p.y) * (r.x - q.x) -
16             (q.x - p.x) * (r.y - q.y);
17     if (val == 0) return 0;
18 }
19
20 int main() {
21     struct Punto p, q, r;
22     printf("Puntos ingresados:\n");
23     p.x = 0; p.y = 3;
24     q.x = 2; q.y = 2;
25     r.x = 1; r.y = 1;
26     printf("Orientación: %d\n", orientacion(p, q, r));
27     return 0;
28 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

=== CALCULO DE CIERRE CONVEXO ===

Puntos ingresados:

(0, 3)

(2, 2)

(1, 1)

(2, 1)

(3, 0)

(0, 0)

(3, 3)

Ln 24, Col 53 Spaces: 4 UTF-8 CRLF Win32

5:25 PM 5/8/2026

```
File Edit Selection View Go Run ... Portafolio_SQL_Junior Update
EXPLORER
PORTAFOLIO_SQL_JUNIOR
  project_08_computational_geometry
  demo_pro.gif
  ejercicio1_full.c
  ejercicio2_full.c
  ejercicio3_full.c
  output_social.mp4
  Triangulacion_Poligonos_full_PRO.docx
  project_09_top_customers_analysis
  query.sql
  README.md
  project_10_monthly_revenue_trend
    cierre_convexo.c
    programa.exe
    query.sql
    README.md
  proyecto_01_creacion_bd
  proyecto_02_consultas_basicas
  proyecto_03_relaciones_joins
  proyecto_04_sales_analysis
  proyecto_05_kpi_report
  05_kpi_and_executive_insights.sql
  proyecto_06_advanced_sql
  proyecto_07_data_warehouse
  .gitignore
  OUTLINE

cierre_convexo.c
22 // Función cierre convexo
23 // Implementa el algoritmo Gift Wrapping (Marcha de Jarvis)
24 // Encuentra los puntos que forman el cierre convexo
25 void cierreConvexo(struct Punto puntos[], int n) {
26     if (n < 3) return;
27
28     int l = 0;
29     for (int i = 1; i < n; i++)
30         if (puntos[i].x < puntos[l].x)
31             l = i;
32
33     int p = l, q;
34
35     printf("Puntos del cierre convexo:\n");
36
37     do {
38         printf("(%d, %d)\n", puntos[p].x, puntos[p].y);
39     } while (p != l);
40 }
41
42 int main() {
43     struct Punto puntos[] = {
44         {0, 3}, {2, 2}, {1, 1}, {2, 1},
45         {3, 0}, {0, 0}, {3, 3}, {2, 4}
46     };
47     int n = 8;
48     cierreConvexo(puntos, n);
49 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

=== CALCULO DE CIERRE CONVEXO ===

Puntos ingresados:

(0, 3)
(2, 2)
(1, 1)
(2, 1)
(3, 0)
(0, 0)
(3, 3)

Ln 24, Col 53 Spaces: 4 UTF-8 CRLF Win32 5:26 PM 5/8/2026

```
File Edit Selection View Go Run ... Portafolio_SQL_Junior Update
EXPLORER
PORTAFOLIO_SQL_JUNIOR
  project_08_computational_geometry
  demo_pro.gif
  ejercicio1_full.c
  ejercicio2_full.c
  ejercicio3_full.c
  output_social.mp4
  Triangulacion_Poligonos_full_PRO.docx
  project_09_top_customers_analysis
  query.sql
  README.md
  project_10_monthly_revenue_trend
    cierre_convexo.c
    programa.exe
    query.sql
    README.md
  proyecto_01_creacion_bd
  proyecto_02_consultas_basicas
  proyecto_03_relaciones_joins
  proyecto_04_sales_analysis
  proyecto_05_kpi_report
  05_kpi_and_executive_insights.sql
  proyecto_06_advanced_sql
  proyecto_07_data_warehouse
  .gitignore
  OUTLINE

cierre_convexo.c
51
52 // FUNCIÓN PRINCIPAL
53 // Función principal:
54 // - Define los puntos de prueba
55 // - Muestra los datos ingresados
56 // - Llama al algoritmo de cierre convexo
57 int main() {
58
59     printf("=== CALCULO DE CIERRE CONVEXO ===\n\n");
60
61     struct Punto puntos[] = {
62         {0, 3}, {2, 2}, {1, 1}, {2, 1},
63         {3, 0}, {0, 0}, {3, 3}, {2, 4}
64     };
65
66     int n = 8;
67     cierreConvexo(puntos, n);
68 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Calculando cierre convexo...

Puntos del cierre convexo:

(0, 3)
(0, 0)
(3, 0)
(3, 3)
(2, 4)

PS C:\Users\mamoy\OneDrive\Documents\Portafolio_SQL_Junior\project_10_monthly_revenue_trend>

Ln 24, Col 53 Spaces: 4 UTF-8 CRLF Win32 5:27 PM 5/8/2026

Figura 1. Ejecución del programa en Visual Studio Code mostrando los puntos del cierre convexo.

Explicación de la actividad desarrollada:

El problema del cálculo del cierre convexo se resolvió mediante la implementación de un algoritmo basado en la técnica de **Gift Wrapping (Marcha de Jarvis)**. Este algoritmo permite identificar los puntos que conforman la envolvente convexa de un conjunto de coordenadas en el plano bidimensional.

El procedimiento inicia seleccionando el punto con menor coordenada en el eje X (o el más bajo en Y), garantizando que este pertenezca al cierre convexo. Posteriormente, el algoritmo evalúa iterativamente el siguiente punto que forma el menor ángulo respecto al punto actual, simulando un recorrido alrededor del conjunto de puntos.

Este proceso se repite hasta regresar al punto inicial, formando así un polígono cerrado que representa el cierre convexo.

Complejidad del algoritmo:

El algoritmo Gift Wrapping (Marcha de Jarvis) presenta una complejidad temporal de $O(n \cdot h)$, donde:

- n = número total de puntos
- h = número de puntos en el cierre convexo

Este algoritmo es eficiente cuando $h \ll n$, es decir, cuando la cantidad de puntos en la envolvente es mucho menor que el total de puntos.

Implementación en C:

El algoritmo fue implementado en lenguaje C utilizando estructuras de datos (struct) para representar los puntos en el plano. Se utilizó un arreglo predefinido de coordenadas para facilitar las pruebas y validar el correcto funcionamiento del algoritmo.

Además, el programa fue compilado y ejecutado utilizando el compilador GCC en el entorno Visual Studio Code, verificando su correcto funcionamiento mediante la visualización de los puntos pertenecientes al cierre convexo en la salida por consola.

Análisis de resultados:

El programa desarrollado permite identificar correctamente los puntos que conforman el cierre convexo. A partir de los datos ingresados, se logra distinguir entre puntos internos (que no pertenecen al perímetro) y puntos externos que forman la envolvente.

Este resultado es fundamental en problemas de geometría computacional, ya que permite simplificar estructuras complejas en representaciones más manejables, optimizando cálculos posteriores como detección de colisiones o análisis espacial.

El algoritmo optimiza el proceso al basarse en evaluaciones de orientación entre puntos, reduciendo la cantidad de comparaciones innecesarias y mejorando la eficiencia en el procesamiento de datos geométricos.

Aplicación en el mundo real:

El cálculo del cierre convexo tiene múltiples aplicaciones en el mundo real. En robótica, se utiliza para la planificación de trayectorias y la evitación de obstáculos, simplificando el entorno en formas convexas para mejorar la eficiencia de los algoritmos.

En gráficos por computadora, particularmente en motores gráficos como OpenGL, se emplea en la construcción de “bounding volumes” que permiten optimizar la detección de colisiones y el renderizado en tiempo real.

Asimismo, en sistemas de información geográfica (GIS), se aplica para delimitar regiones, analizar distribuciones espaciales y optimizar consultas de ubicación.

La abstracción geométrica permite transformar datos complejos en estructuras simples, facilitando la toma de decisiones computacionales en tiempo real.

Conclusión

La implementación del algoritmo de cierre convexo permite resolver de manera eficiente problemas de delimitación espacial en el plano cartesiano. A través del uso de técnicas algorítmicas como **Gift Wrapping**, se logra identificar la envolvente mínima que contiene un conjunto de puntos.

Este tipo de soluciones demuestra la importancia de la geometría computacional en aplicaciones reales, donde la optimización del procesamiento y la reducción de complejidad son factores clave.

Finalmente, la traducción de conceptos matemáticos a código funcional en lenguaje **C** fortalece las habilidades del estudiante en análisis algorítmico, estructuras de datos y resolución de problemas reales.

Bibliografía

1. Altamirano Briceño, Y. A. (2026). *Material de Geometría Computacional (Folletos 8 y 9)*. Universidad Nacional de Ingeniería, Nicaragua.
2. Preparata, F. P., & Shamos, M. I. (1985). *Computational Geometry: An Introduction*. Springer.
3. de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications*. Springer.
4. GeeksforGeeks. (2024). *Convex Hull Algorithm (Jarvis March)*. Recuperado de: <https://www.geeksforgeeks.org/convex-hull-using-jarvis-algorithm/>
5. The Coding Train. (2019). *Gift Wrapping Algorithm (Convex Hull)* [Video]. YouTube.